

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Phase - 2 Report

Authors:

Name	Id	Contribution
RAJESH A S	2024ac05683@wilp.bits-pilani.ac.in	100%
Pavan Kumar S	2024ac05682@wilp.bits-pilani.ac.in	100%
DineshKumar P	2024ac05562@wilp.bits-pilani.ac.in	100%
Sachin Deepak K	2024ac05608@wilp.bits-pilani.ac.in	100%

Links:

Source code Github: <https://github.com/PavanKumar-WILP/logibridge.git>

Demo Video:

https://drive.google.com/drive/folders/19H4s_0iOVaI6ZdRAbyKI5efIC0zoU3Wt?usp=sharing

Table of Contents

Phase 2 Report.....	3
1 Component C – Sensor Pipeline and MQTT Architecture.....	3
1.1 Task C1 – Sensor Simulator.....	3
1.2 Task C2 – Pre processing Pipeline.....	7
1.3 Task C3 – Data Fusion Justification.....	11

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Phase 2 Report

1 Component C – Sensor Pipeline and MQTT Architecture

1.1 Task C1 – Sensor Simulator

Objective

The objective of the sensor simulator is to emulate the real-time operation of a refrigerated pharmaceutical truck by continuously generating realistic sensor readings. Instead of requiring physical hardware during development, the simulator creates synthetic sensor streams that closely resemble operational conditions and publishes them to a **local Mosquitto MQTT broker**. This allows the complete Edge AI pipeline—including preprocessing, feature extraction, inference, and alert generation—to be tested under both normal and abnormal operating conditions.

Simulated Sensor Streams

Three independent sensor streams are generated.

1. Temperature Sensor

The refrigerated compartment is configured with a target temperature of **4°C**.

During normal operation, the temperature follows a Gaussian (normal) distribution:

$$T \sim N(4.0^{\circ}\text{C}, 0.3^{\circ}\text{C})$$

where:

- Mean = 4.0°C
- Standard deviation = 0.3°C
- Sampling frequency = **1 Hz**

This represents minor fluctuations caused by compressor cycling and environmental influences.

Temperature Drift Anomaly

When the simulator is executed using "--anomaly temp_drift" the temperature increases gradually by 0.08 degree Celsius every reading.

Therefore, $T_{new} = T_{previous} + 0.08$

This models compressor degradation where cooling efficiency slowly decreases before complete failure.

2. Vibration Sensor

The refrigeration compressor vibration is measured using a three-axis accelerometer.

Normal vibration: $V \sim N(0.45g, 0.05g)$

Sampling frequency: **0.5 Hz**

The vibration remains relatively stable while the compressor operates normally.

Bearing Wear Anomaly

Running --anomaly vibration

changes the vibration distribution to $V \sim N(1.2 g, 0.15 g)$

This sudden increase represents excessive compressor vibration caused by bearing wear or mechanical imbalance.

3. Door Event Sensor

The door sensor generates discrete MQTT events.

Possible values: OPEN or CLOSE

along with

- Timestamp
- Truck ID

Door events occur only when loading or unloading cargo.

Combined Failure Mode

Running “--anomaly combined”

simultaneously activates

- Temperature drift
- High compressor vibration

This represents a severe refrigeration failure requiring immediate intervention.

MQTT Publishing Architecture

MQTT topic hierarchy used for communication between edge devices and the LogiEdge cloud platform.

```
freightbridge/  
└─ truck/  
   └─ {truck_id}/  
      ├── telemetry/  
      │   ├── temperature  
      │   ├── humidity  
      │   ├── vibration  
      │   └─ gps  
      ├── inference/  
      │   ├── prediction  
      │   └─ confidence  
      ├── alerts/  
      │   ├── temperature_breach  
      │   ├── vibration_alert  
      │   └─ sensor_fault  
      ├── command/  
      │   ├── update_model  
      │   ├── restart  
      │   └─ config  
      └─ status/  
          ├── heartbeat  
          └─ connection
```

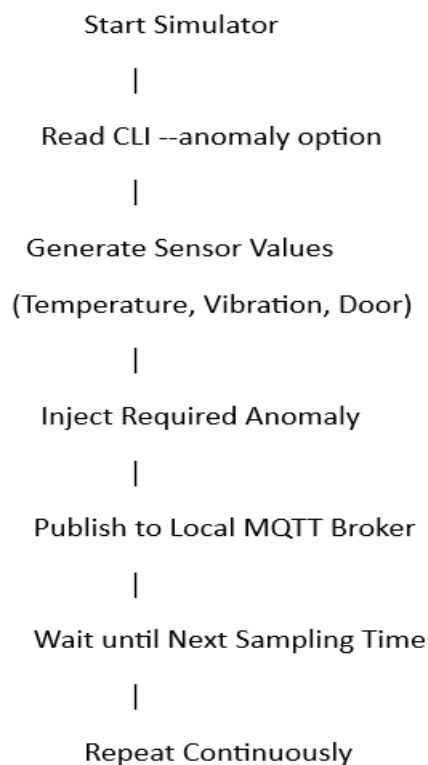
All simulated sensor values are published to a **local Mosquitto MQTT broker** running on

Localhost

QoS justification

MQTT Topic	QoS	Justification
telemetry/	QoS 0	Temperature, humidity, vibration and GPS data are published periodically. Since newer readings quickly replace older ones, occasional packet loss is acceptable. QoS 0 minimizes latency and bandwidth usage, making it suitable for continuous sensor streams.
inference/	QoS 1	Model predictions and confidence scores should be delivered reliably for fleet monitoring. Duplicate messages can be tolerated by the application, while QoS 1 ensures reliable delivery with moderate communication overhead.
alerts/	QoS 1	Temperature breaches, excessive vibration and sensor faults are safety-critical events that must not be missed. QoS 1 guarantees message delivery while avoiding the additional overhead of QoS 2.
command/	QoS 1	Commands such as model updates, configuration changes and device restart must reliably reach the edge device. Duplicate command messages are handled safely through idempotent execution.
status/	QoS 0	Heartbeat and connection messages are transmitted periodically. Missing a single heartbeat is acceptable because the next update arrives shortly, making low communication overhead preferable.

Simulator Workflow



Simulator Usage

The following command line options can be used while using the simulator.py

Command Line Usage:

Command	Purpose
python simulator.py --anomaly none	Normal operation
python simulator.py --anomaly temp_drift	Temperature anomaly
python simulator.py --anomaly vibration	Compressor anomaly
python simulator.py --anomaly combined	Critical failure

1.2 Task C2 – Pre processing Pipeline

Objective

Raw sensor data contain measurement noise and cannot be used directly for machine learning. The preprocessing pipeline converts continuous sensor streams into normalized feature vectors suitable for real-time classification.

Preprocessing Rationale

- Filtering: Raw temperature (1 Hz) and vibration RMS (0.5 Hz) pass through a 5-sample moving average filter to suppress high-frequency noise.
- Sliding Windowing: Features are computed over a 30-second sliding window with a 10-second step size (66.6% overlap).
- Feature Extraction: Each window is transformed into a 6-value feature vector: Temperature Mean, Temperature Std, Temperature Rate-of-Change, Vibration RMS, Vibration Peak, and Vibration Kurtosis.

The implemented pipeline follows the exact sequence specified in the project requirements.

Step 1 – Moving Average Filtering

A **5-sample moving average filter** is applied independently to both temperature and vibration streams.

The moving average is computed as

$$MA = (x_1 + x_2 + x_3 + x_4 + x_5) / 5$$

Purpose:

- Remove sensor noise
- Reduce random spikes
- Produce smoother signals
- Improve feature stability

Door events remain unchanged because they are discrete values.

Step 2 – Sliding Window Feature Extraction

Filtered sensor readings are grouped into

- Window length = **30 seconds**
- Step size = **10 seconds**

This creates overlapping windows that allow the model to detect gradual changes without waiting for a completely new data sequence.

Each window generates six numerical features.

Temperature Features

1. Mean temperature

$$\text{Mean} = \mu = (\sum T_i) / n$$

Represents average cargo temperature.

2. Standard deviation

$$\sigma = (\text{Sqrt} \sum (T_i - \mu)^2) / 2$$

Measures temperature stability.

3. Rate of change

$$\text{ROC} = (T_{\text{last}} - T_{\text{first}}) / \Delta t$$

Expressed in °C/min

Positive values indicate warming.

Vibration Features

4. RMS vibration

$$\text{RMS} = \sqrt{\sum V_i^2 / n}$$

Measures compressor operating intensity.

5. Peak vibration

Maximum vibration value observed within the window. Useful for detecting sudden shocks.

6. Kurtosis

Measures the sharpness of the vibration distribution. Higher kurtosis indicates abnormal vibration spikes associated with mechanical failures.

Generated Feature Vector

Each inference window produces

[

Temp Mean,

Temp Std,

Temp Rate,

Vib RMS,

Vib Peak,

Vib Kurtosis

]

This six-dimensional feature vector becomes the input to the neural network classifier.

Step 3 – Feature Normalization

Machine learning models perform better when input features have similar numerical ranges.

The project requires that normalization statistics be computed **only once** using **10 minutes of clean Normal-class data**.

For each feature, Mean and Standard deviation are calculated

Normalized feature: $x' = (x - \mu) / \sigma$

The computed statistics are stored as training_stats.npy

During inference, training_stats.npy is loaded.

Live data are **never used** to recompute statistics.

This guarantees identical preprocessing during both training and deployment.

Mandatory Normalization Experiment

The project requires two experiments.

Experiment 1:

Using Correct training_stats.npy

Expected result:

High classification accuracy because training and inference distributions match.

Experiment 2

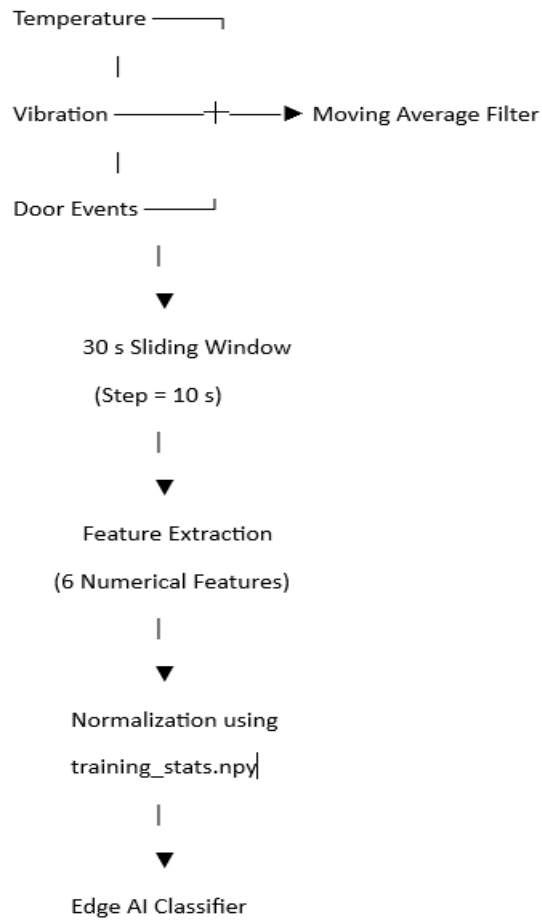
Artificially shift every mean by 3 Sigma (3σ)

Expected observations:

- Incorrect normalization
- Feature values become distorted
- Decision boundaries shift
- Increased class confusion
- Significant reduction in classification accuracy

This demonstrates the importance of preserving training statistics throughout deployment.

Overall Pre-processing Pipeline



Normalization Impact experiment results	
Normalization Parameter Set	Validation Classification Accuracy (%)
Baseline Stats (training_stats.npy)	94.74%
Artificially Shifted (+3 σ Stats)	91.23%

1.3 Task C3 – Data Fusion Justification

Selected Fusion Method

The proposed LogiEdge system implements **feature-level fusion**.

Each sensor is processed independently to extract meaningful characteristics. The extracted features are then concatenated into a single six-dimensional feature vector before classification.

Significance of Feature-Level Fusion:

Feature-level fusion combines complementary information from multiple sensors before machine learning inference.

For example:

Temperature provides information about cooling performance.

Vibration indicates compressor health.

Door events explain temporary temperature increases caused by cargo loading.

By combining these features, the classifier gains a complete understanding of truck operating conditions.

Comparison with Data-Level Fusion

Data-level fusion combines raw sensor values directly before preprocessing.

For this application, data-level fusion is not appropriate because:

- Sensors operate at different sampling frequencies (temperature: **1 Hz**, vibration: **0.5 Hz**, door events: discrete).
- Raw measurements have different units (°C, g, and event states).
- Synchronizing heterogeneous sensor streams increases computational complexity and memory usage on the edge device.
- Noise from one sensor may negatively influence the combined raw signal.

Comparison with Decision-Level Fusion

Decision-level fusion trains separate classifiers for each sensor and combines their predictions.

Although this approach improves fault tolerance, it has several disadvantages:

- Multiple models increase memory usage and inference time.
- Conflicting predictions require additional fusion logic.
- More computational resources are needed, increasing power consumption.
- Maintaining multiple models complicates deployment and OTA updates.

Justification for FreightBridge

Feature-level fusion offers the best trade-off between accuracy, computational efficiency, and implementation complexity for the FreightBridge deployment.

It enables:

- Integration of complementary temperature and vibration information.
- Low computational overhead suitable for edge hardware.
- A single lightweight neural network model.
- Faster inference while remaining within the **90-second detection requirement**.
- Simplified deployment, maintenance, and OTA model updates.

Therefore, feature-level fusion is the most suitable architecture for real-time pharmaceutical cold-chain monitoring, balancing detection accuracy with the resource constraints of an edge device.

Data Fusion Justification

The proposed LogiEdge system uses **feature-level fusion**. Features are first extracted independently from each sensor stream. The temperature sensor contributes three statistical features (mean, standard deviation, and rate of change), while the vibration sensor contributes three features (RMS, peak value, and kurtosis). These six features are concatenated into a single joint feature vector and supplied as the input to the neural network classifier.

Feature-level fusion was selected because it provides an effective balance between computational efficiency and predictive performance for edge deployment. By extracting informative statistical features before fusion, the dimensionality of the data is significantly reduced while preserving the characteristics required to distinguish between normal, warning, and critical operating conditions. This approach is well suited for embedded inference using a lightweight MLP and INT8 TensorFlow Lite model.

Data-level fusion was not selected because it would require combining raw temperature and vibration time-series data before feature extraction. Since the sensors operate at different sampling frequencies (1 Hz for temperature and 0.5 Hz for vibration), additional synchronization and alignment logic would be required, increasing computational complexity and memory usage on the edge device.

Decision-level fusion was also not selected because it would require separate models for each sensor followed by an additional decision fusion stage. This would increase model size, inference latency, and maintenance complexity while providing little additional benefit for the relatively small number of sensor modalities used in this application.

Therefore, feature-level fusion offers the most appropriate trade-off between computational cost, implementation simplicity, and classification accuracy for real-time cold-chain monitoring on resource-constrained edge devices.

